Bring the BIM judgment.

SWACCA BIM Summit 2026 | Two one-hour sessions

You do not need to become a software developer to direct useful work. You do need to explain the problem, define what a trustworthy answer looks like, and ask the agent to show its evidence.

The example we will follow

Can we tell whether required wall and ceiling type information is present? We turn a written checklist into a read-only Revit check, add an Excel report, and connect the same calculation to desktop AI tools.

Minutes	Session 1: from BIM question to researched plan
00-10	Frame the work and preview the example.
10-18	Direct the agent with Git: inspect, branch, diff, commit.
18-33	Research existing work, resources, dependencies and licenses.
33-43	Define rules, scope and independently checkable sample cases.
43-55	Show the first working step, including quick scripting routes.
55-60	Questions.

Minutes	Session 2: from working example to your next tool
00-05	Reconnect the BIM question to the declared scope.
05-23	Show how we built the plugin and its MCP connection.
23-35	Verify, debug and learn from independent review.
35-50	Use bim-agent-lab afterward with your own agent.
50-60	Choose an office use case, then questions.

This is a guided account of a build, with selected demonstrations. The repository holds the detailed procedures for later. Inventory supports the readiness answer; measured quantities and physical-condition checks are future extensions.

One repetitive BIM question I would like to answer:

Start with a useful question.

Your responsibility is the purpose, scope and standard of evidence.

Include	For our example
Purpose	Find missing type information before a defined handoff.
Scope	The declared host walls and ceilings, level and phase; say whether links count.
Rules	Which fields are required for which types, and which exceptions are explicit.
Boundaries	Read a disposable sample. Do not change model values.
Output	Findings, affected instances, coverage and a clear scoped verdict.
Proof	Known passing, failing and incomplete cases; compare with Revit schedules.

Suggested prompt: frame the task before building

I want to check [information] before [handoff or decision]. Read the project context first. Help me define the scope, rules, exceptions and expected output. Research existing approaches and required resources. Propose a small first step and explain how we can verify it independently. Identify what you need me to decide before writing code.

Make the division of work explicit

You supply professional intent, choose acceptable behavior and judge whether the answer is useful. The agent researches, inspects source, proposes a plan, writes code, runs checks and records evidence. Ask it to distinguish what it observed from what it assumes.

For desktop demonstrations, the person handles Revit menus and permission prompts. Ask for one action at a time, with an exact file path. Opening a similarly named model or switching to another view is not proof that the intended document is active.

A useful follow-up

Show me the source or test that supports that claim. What remains unverified? What small check would resolve it?

My intended scope and one result I can check without trusting the agent:

Ask it to look first.

Collect the resources and inspect the actual implementation.

An API is the interface a program exposes. An SDK supplies development tools and libraries. A dependency is another package our code needs. A repository holds files and their tracked history. These are resources the agent should identify before it starts building.

Suggested prompt: research and collect

Find relevant official APIs, SDKs, libraries and existing GitHub projects. Check current versions, compatibility and licenses. Clone promising repositories into an ignored research folder and inspect the code behind the feature. Record source links and exact revisions. Distinguish a dependency we consume, code we adapt and a precedent we only learn from.

In our research, source inspection exposed a default query limit of 100 elements. That matters: a partial inventory cannot silently represent a whole model. A project description alone did not answer that question.

Git term	What you are asking the agent to do
Clone	Make a local copy so it can inspect an existing repository.
Branch	Name a line of work while preserving existing changes.
Diff	Show exactly what changed and explain its effect.
Commit	Record a selected, checked checkpoint in local history.
Remote / push	Identify the hosted destination; transfer commits to it.
Commit ID	Identify the exact version that was inspected or tested.

Suggested prompt: make the work reviewable

Inspect the current repository, branch and changes. Preserve existing work. Use an appropriate branch, show the diff and relevant checks, then commit a coherent step. Keep credentials, research clones and working model files out of Git. Confirm the destination and visibility before sharing.

Ask for versions at every boundary

Revit version, SDK package version, MCP protocol revision and desktop app version are different facts. Ask what works together now, then pin and test that combination. A newer SDK does not prove that every client supports its newest protocol.

Try one small read first.

The agent writes the script; you run it in the correct Revit environment.

Dynamo: a script inside a graph

Dynamo for Revit is included with Revit 2024. For this beginner example, its CPython3 engine supplies its own Python runtime; a separate Python installation is not needed. Ask the agent to confirm your installed Revit, Dynamo and engine versions first. [1, 2]

1. Open a disposable sample project. In Revit, choose Manage > Visual Programming > Dynamo; create a new graph.

2. Set run mode to Manual. Search for a Python Script node, place it, select CPython3 and open its editor.

3. Replace the editor contents with the two lines below. Accept the edit, connect OUT to a Watch node and click Run. Save the graph for reuse.

```
import sys
OUT = ["Dynamo is working", sys.version]
```

This first check only reports the Python version. Next, ask the agent for a read-only wall/ceiling inventory with a clearly stated scope, and compare its count with a native Revit schedule.

pyRevit: a script behind a ribbon button

pyRevit is a separate, optional Revit add-in. It can run Python scripts packaged as ribbon tools. The presenter has installed it; it is not an added participant prerequisite for following the seminar. [3]

1. Ask the agent to inspect the installed pyRevit version and Python engine, then create a small extension containing a read-only script.py command.

2. Have the agent give the exact extension-folder registration and reload steps for that version. Run the new ribbon button on the same sample and verify the output.

Suggested prompt for either route

For my installed [Dynamo or pyRevit] environment, create a read-only inventory of host basic walls and ceilings by type. State the scope and exclusions. Give me the exact run steps and an independent count check. Do not edit or save the model, and explain any extra dependency before adding it.

Dynamo and pyRevit have different host objects and may use different Python engines. Ask for code adapted to the selected route; do not assume the same script can be pasted into both. These short demos precede the compiled-plugin build.

Zero findings is not enough.

A readiness verdict needs both the rules and complete coverage.

Our teaching scope contains 16 declared instances across six types: 12 walls and four ceilings. Every type requires Code (Type Mark). Walls A and B also require Fire Rating and Assembly Reference. Other exemptions are written into the checklist; the tool does not infer them from blank values.

Prepared case	Missing findings	Affected instances	Coverage complete	Evaluation complete
Complete	0	0	True	True
Wall defects	2	6	True	True
Ceiling negative	3	8	True	True
NC1: missing element	0	0*	False	True
NC2: missing rule	0	0*	False	False

*Affected instances counts missing-value findings only. NC1 examines 15 of 16: one declared identity is absent. NC2 finds all 16, but two Ceiling ACT instances cannot be evaluated because their type has no matching requirement. Neither case is a readiness pass.

Suggested prompt: ask for an assessable answer

Get a fresh readiness context and check required wall and ceiling type information. Report the full model path, scope, requested and examined counts, findings and affected instances, both completeness flags, scope issues and unexamined cases. Explain whether the declared scope passes. Do not edit or export anything.

Check the answer against something independent

Use native Revit schedules and a prepared answer sheet. Confirm the intended model is active. A missing field on one type may affect several placed instances. An instance count is not an area, length or material quantity.

Keep the claim inside its scope

The tool checks whether required information is present. It does not validate a fire-rated assembly, certify compliance or establish whole-project readiness. Other host elements and links are excluded from this teaching scope.

A context token binds a request to the observed document and checklist. After a model switch, a stale token should be rejected. Acquire a fresh context to recover; an expired-token test alone does not prove model-switch detection.

Build in checkable steps.

Keep the calculation separate from the way people ask for it.

Step	How we engaged the agent
Prove the setup	Check prerequisites; build a minimal read-only Revit command.
Add one useful rule	Define required type information and deliberately controlled sample results.
Add Excel output	Review a library and its dependencies; compare the workbook with known results.
Connect an agent	Compare existing bridges with a small custom bridge; test actual desktop compatibility early.
Prepare recovery	Record installed files and backups before changes; test restoration.
Retain the lesson	Record participant decisions and the agent's reproducible procedure.

Our first dependency: ClosedXML

The author asked for an Excel-readable result. We chose ClosedXML and inspected its resolved dependencies and licenses. The workbook contains Summary, Findings, Affected elements, and Rules and coverage. A passing build did not predict the first Revit loading failure; host testing led to a separate exporter process.

Suggested prompt: add a dependency responsibly

Add [output] using a suitable library. Research its current compatible version, license and transitive dependencies. Explain what will be installed. Verify the output against our independent sample results in the actual application, then record the process in both teaching layers.

What MCP adds

Desktop AI app > **MCP server** > **Revit add-in**

MCP (Model Context Protocol) lets an AI application discover and call tools. Our server exposes a fresh-context tool and a readiness-check tool. The add-in reads Revit through its supported API context; the shared evaluator applies the written rules.

Two ways to use the same readiness calculation

The manual Revit command saves JSON/text evidence and the Excel workbook. The MCP tools return structured results to the agent; those calls do not edit the model or export files.

The bridge consumes official C# MCP SDK 2.2.0. The TypeScript SDK 1.26.0 was a learning precedent, not copied implementation code. This is a recorded build baseline, not a claim about today's latest versions.

Ask what connects the findings.

A reviewer can improve the reasoning as well as identify defects.

Before the larger bridge build, we asked the agent for a written specification and implementation plan. A separate reviewer inspected each. Reviewer closure and the author's approval were distinct steps. Implementation review followed with source, tests and evidence tied to exact commits.

Two or three major findings are a signal

Ask whether the defects share an underlying missing concept or framework. The count is a cue to investigate, not proof. Give the explanation back to the coding agent so it can address the broader problem rather than patch only the named examples.

Suggested prompt to the independent reviewer

Do these findings share a missing concept or reasoning framework? Explain the connection with evidence, identify other places it may apply, and state the tests that would demonstrate the broader guarantee. Distinguish a confirmed pattern from a hypothesis.

What that revealed in our build

Repeated cancellation findings exposed a gap between keeping resources safe and allowing shutdown to finish. The coder had reasoned about who owned objects, but not fully about which work waited for other work. Tests then interrupted real operations at their handoffs, including callbacks that failed or stayed running.

Suggested prompt back to the coding agent

Assess the reviewer's diagnosis against the source before patching. State the guarantee that must hold across the whole operation, including interruption and failure. Apply the supported insight to every affected handoff, derive tests from it, preserve the original reproductions, and explain why the correction covers more than those examples.

Ask what each kind of evidence proves

A build proves compilation. Offline tests exercise specified behavior. A live Revit run tests the host integration. A desktop answer tests what the participant actually sees. Freeze the candidate and record its identity; evidence from an older installed build does not automatically prove the current source.

A question I will ask before accepting an agent's work:

Continue with bim-agent-lab.

Use the repository as both a refresher and context for your agent.

github.com/bringfire/bim-agent-lab

Scan or type the address. Private during preparation; participant access will be announced at the seminar.

Start with	Use it for
README.md	Current status and navigation.
docs/start-here.md	Your first steps as a participant.
docs/tutorials/README.md	How we engaged the agent at each stage.
AGENTS.md	Rules and boundaries for agents working here.
docs/how-to/agent-handoff.md	Technical setup, verification and remaining work.

Suggested first prompt in your own checkout

Clone https://github.com/bringfire/bim-agent-lab into a new folder. Read README.md, AGENTS.md, docs/start-here.md and docs/how-to/agent-handoff.md. Explain what works and what remains unverified. Inspect my prerequisites and guide me through one useful step. Tell me which Revit actions I must take. Explain proposed installations or settings changes before making them.

Reading the lessons requires no Revit installation. Running the example requires the documented Revit environment and separately acquired resources. Autodesk sample model binaries and API libraries are not bundled. Check the current setup and acceptance status before installing; this packet is not a release certification.

My next small BIM question, and how I will verify the answer:

Official resources for the quick demos

[1] Autodesk: Dynamo for Revit 2.19.3 (Revit 2024)

[2] Dynamo: Python 3 installation and embedded runtime

[3] pyRevit: project, installation and extension structure

The digital PDF links these sources. Full URLs, preparation notes and the editable packet live in workshops/swacca-2026/handout/. Research checked October 23, 2026. Reusable prompts here are suggestions, not a reconstructed transcript.